

Install and compile MPAS-Urban

Guide 1 of 3 | LIU Zhuo | The Hong Kong University of Science and Technology

This guide builds the standard HKUST-MPAS `hkust-dev` version. Work in a Linux Bash shell. Use one compiler/MPI/NetCDF stack for both compilation and execution.

Step 1 | Prepare the environment

On a cluster, load the site's GNU compiler, MPI, NetCDF-C, NetCDF-Fortran and PnetCDF modules. Module names are machine-specific. If these tools are unavailable, complete [Appendix A: environment installation commands](#), also included at the end of this PDF. It covers the basic tools, MPI, all required libraries, METIS and restoring the environment after login.

```
export MPAS_ROOT="$HOME/MPAS"
mkdir -p "$MPAS_ROOT"
command -v gcc gfortran mpicc mpif90 mpiexec
command -v nc-config nf-config pnetcdf-config
```

Each command must be found.

You also need METIS `gpmets` to partition the mesh in Guide 2. Load your site's METIS module or use the Ubuntu installation command in Appendix A, then check:

```
command -v gpmets
```

Set the MPAS build variables:

```
export NETCDF="$(nc-config --prefix)"
export NETCDFF="$(nf-config --prefix)"
export PNETCDF="$(pnetcdf-config --prefix)"
unset PIO
test -s "$NETCDF/include/netcdf.h"
test -s "$NETCDFF/include/netcdf.mod"
test -s "$PNETCDF/include/pnetcdf.h"
```

`unset PIO` removes any PIO setting inherited from an earlier session. This workflow uses the SMIOOL library bundled with MPAS, together with PnetCDF; it does not require a separate PIO2 installation.

Exports apply to the current shell only. After logging in again, reload your modules or the Appendix A library paths, then repeat the exports. Do the same in your batch job's environment.

Step 2 | Download HKUST-MPAS and this tutorial

```
cd "$MPAS_ROOT"
git clone --depth 1 --branch hkust-dev --single-branch \
  https://github.com/HKUST-MPAS/HKUST-MPAS.git
git clone https://github.com/Liuzh223/MPAS-Urban-Tutorial.git
export MPAS_BUILD="$MPAS_ROOT/HKUST-MPAS"
cd "$MPAS_BUILD"
git rev-parse HEAD
```

Record the printed commit with your experiment. `MPAS_BUILD` means the source directory containing your compiled executables. This tutorial does not require a separate modified source tree. Access to this tutorial's repository is required while it remains private.

Step 3 | Compile initialization

```
cd "$MPAS_BUILD"  
make -j 1 gfortran CORE=init_atmosphere PRECISION=single  
test -x init_atmosphere_model
```

Successful compilation produces `init_atmosphere_model` in the source directory. Stop here if compilation fails.

Step 4 | Compile the atmosphere model

```
make clean CORE=atmosphere  
make -j 1 gfortran CORE=atmosphere PRECISION=single  
test -x init_atmosphere_model  
test -x atmosphere_model  
ls -lh init_atmosphere_model atmosphere_model
```

The `clean` command targets the atmosphere core and retains the initialization executable. Use `make -j 1` for both cores because parallel compilation may cause Fortran module dependency errors on some systems.

Step 5 | Check runtime files

```
ls streams.init_atmosphere streams.atmosphere  
ls stream_list.atmosphere.*  
ls -lh vertical_levels/urban_ZR_75.txt  
ls -lh *TBL *DATA*
```

Keep the runtime tables supplied by this source version. Do not replace them with another experiment's tables. Guide 3 links these files into the case directory.

Both executables must exist before moving to Guide 2. Downloading the source alone is not enough to run MPAS.

References

- [HKUST-MPAS source and urban-model instructions](#)
- [Official MPAS tutorial](#)

Appendix A: Install the build and runtime environment

Use this appendix only when the server does not already provide a compatible GNU compiler, MPI, NetCDF, and PnetCDF environment. On a managed HPC cluster, the modules supported by the site are usually the better choice.

Run all commands in this appendix in the **same shell session**. Every MPI-linked package must use the same GNU compiler family and MPI wrappers. If you open a new terminal or log in again, repeat the environment exports in [Create the local installation prefix](#). If you build Open MPI below, also repeat its final PATH and LD_LIBRARY_PATH exports.

The examples use these pinned versions:

- Open MPI 4.1.6
- zlib 1.3.1
- HDF5 1.14.6
- NetCDF-C 4.9.3
- NetCDF-Fortran 4.6.4
- PnetCDF 1.12.3

These versions form one tested example stack. They are not a requirement to use the newest releases. Changing several versions at once makes build failures harder to diagnose.

The required library build order is:

1. zlib
2. HDF5
3. NetCDF-C
4. NetCDF-Fortran
5. PnetCDF

Stop if any `make check`, `file check`, or `prefix check` fails. Continuing with a partially installed stack usually causes a less obvious MPAS compile or runtime error later.

Install the compiler and basic tools

The following commands are for Ubuntu 22.04 and require administrator privileges. On a managed cluster without `sudo`, load the site's compiler module and ask the administrator for any missing tools instead.

```
sudo apt-get update
sudo apt-get install -y \
  build-essential gfortran git curl ca-certificates \
  m4 perl tar gzip bzip2 xz-utils nano
```

`build-essential` provides GCC, G++ and Make; `gfortran` provides the Fortran compiler. The remaining tools download, unpack, configure and edit the source files. See the [Ubuntu GNU Fortran package](#).

```
gcc --version
g++ --version
gfortran --version
make --version
command -v git curl m4 perl tar nano
```

Install METIS for mesh partitioning

On Ubuntu, install the partitioning tools and check for gpmets:

```
sudo apt-get install -y metis
command -v gpmets
```

On a cluster, use its METIS module or ask the administrator to install it. METIS is used for mesh partitioning, not linked into the MPAS executable.

Create the local installation prefix

The remaining MPI and library builds install under user-owned directories and do not need sudo.

MPAS_LIB_SRC stores downloaded library sources, while MPAS_PREFIX stores the installed headers, libraries, and configuration tools.

```
export MPAS_ROOT="$HOME/MPAS"
export MPAS_PREFIX="$MPAS_ROOT/opt/mpas-libs"
export MPAS_LIB_SRC="$MPAS_ROOT/src/mpas-libs"
mkdir -p "$MPAS_PREFIX" "$MPAS_LIB_SRC"

export PATH="$MPAS_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LD_LIBRARY_PATH:-}"
export CPATH="$MPAS_PREFIX/include:${CPATH:-}"
export LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LIBRARY_PATH:-}"
export PKG_CONFIG_PATH="$MPAS_PREFIX/lib/pkgconfig:${PKG_CONFIG_PATH:-}"
export PKG_CONFIG_PATH="$MPAS_PREFIX/lib64/pkgconfig:$PKG_CONFIG_PATH"
export CPPFLAGS="-I$MPAS_PREFIX/include"
export LDFLAGS="-L$MPAS_PREFIX/lib -L$MPAS_PREFIX/lib64"

gcc --version
g++ --version
gfortran --version
make --version
```

Do not continue until all four commands print valid version information.

Optional: build Open MPI 4.1.6

Build Open MPI only if the server does not already provide compatible mpicc, mpif90, and mpiexec commands. If the server provides a supported MPI module, load that module and skip this section.

```

export MPAS_ROOT="$HOME/MPAS"
export MPI_PREFIX="$MPAS_ROOT/opt/openmpi-4.1.6"
mkdir -p "$MPAS_ROOT/src" "$MPI_PREFIX"
cd "$MPAS_ROOT/src"

curl -fL -O https://download.open-mpi.org/release/open-mpi/v4.1/openmpi-4.1.6.tar.gz
tar -xzf openmpi-4.1.6.tar.gz
cd openmpi-4.1.6

./configure --prefix="$MPI_PREFIX" CC=gcc CXX=g++ FC=gfortran
make -j 1
make install

export PATH="$MPI_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPI_PREFIX/lib:${LD_LIBRARY_PATH:-}"

mpicc --showme:command
mpif90 --showme:command
mpiexec --version

```

The wrapper checks must show the intended GNU compilers. Keep these Open MPI exports active while building PnetCDF and MPAS. `--showme:command` is an Open MPI option; for MPICH wrappers use `-show` instead, including in the final checks below.

Build zlib 1.3.1

zlib provides compression support used by HDF5 and NetCDF.

```

cd "$MPAS_LIB_SRC"
curl -fL -O https://zlib.net/fossils/zlib-1.3.1.tar.gz
tar -xzf zlib-1.3.1.tar.gz
cd zlib-1.3.1

./configure --prefix="$MPAS_PREFIX"
make -j 1
make check
make install

test -s "$MPAS_PREFIX/include/zlib.h"

```

Build HDF5 1.14.6

HDF5 provides the NetCDF-4 storage layer. MPAS needs the HDF5 C library here; the separate HDF5 Fortran interface is not required by NetCDF-Fortran.

```

cd "$MPAS_LIB_SRC"
curl -fL -O https://github.com/HDFGroup/hdf5/releases/download/hdf5_1.14.6/hdf5-1.14.6.tar.gz
tar -xzf hdf5-1.14.6.tar.gz
cd hdf5-1.14.6

CC=gcc ./configure \
  --prefix="$MPAS_PREFIX" \
  --with-zlib="$MPAS_PREFIX" \
  --enable-shared \
  --disable-fortran

make -j 1
make check
make install

h5cc -showconfig | head -n 12

```

Build NetCDF-C 4.9.3

NetCDF-C provides `netcdf.h` and `libnetcdf`. The final NetCDF-4 feature check must print yes.

```

cd "$MPAS_LIB_SRC"
curl -fL -O https://downloads.unidata.ucar.edu/netcdf-c/4.9.3/netcdf-c-4.9.3.tar.gz
tar -xzf netcdf-c-4.9.3.tar.gz
cd netcdf-c-4.9.3

CC=gcc ./configure \
  --prefix="$MPAS_PREFIX" \
  --disable-dap \
  --enable-netcdf-4 \
  --enable-shared

make -j 1
make check
make install

nc-config --version
nc-config --has-nc4
test -s "$MPAS_PREFIX/include/netcdf.h"

```

Build NetCDF-Fortran 4.6.4

NetCDF-Fortran provides `netcdf.mod` and `libnetcdff`. It must use the NetCDF-C installation under the same `MPAS_PREFIX`.

```

cd "$MPAS_LIB_SRC"
curl -fL -O https://downloads.unidata.ucar.edu/netcdf-fortran/4.6.4/netcdf-fortran-4.6.4.tar.gz
tar -xzf netcdf-fortran-4.6.4.tar.gz
cd netcdf-fortran-4.6.4

CC=gcc FC=gfortran ./configure \
  --prefix="$MPAS_PREFIX" \
  --enable-shared

make -j 1
make check
make install

nf-config --version
test -s "$MPAS_PREFIX/include/netcdf.mod"

```

Build PnetCDF 1.12.3

PnetCDF provides parallel classic-NetCDF I/O. Build it with the same mpicc and mpif90 wrappers that will compile MPAS.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://parallel-netcdf.github.io/Release/pnetcdf-1.12.3.tar.gz
tar -xzf pnetcdf-1.12.3.tar.gz
cd pnetcdf-1.12.3

CC=mpicc FC=mpif90 ./configure \
  --prefix="$MPAS_PREFIX"

make -j 1
make check
make install

pnetcdf-config --version
test -s "$MPAS_PREFIX/include/pnetcdf.h"
```

Export and verify the completed dependency stack

The three configuration tools below identify the exact NetCDF-C, NetCDF-Fortran, and PnetCDF prefixes built above. Exporting all three variables prevents values left by an older shell environment from being reused. PIO remains unset so the MPAS Makefile selects the SMIOL implementation bundled with the source tree.

```
export PATH="$MPAS_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LD_LIBRARY_PATH:-}"

export NETCDF="$(nc-config --prefix)"
export NETCDFF="$(nf-config --prefix)"
export PNETCDF="$(pnetcdf-config --prefix)"

# An unset PIO variable tells MPAS to use its bundled SMIOL source.
unset PIO

command -v mpicc mpif90 nc-config nf-config pnetcdf-config
mpicc --showme:command
mpif90 --showme:command
nc-config --version
nf-config --version
pnetcdf-config --version

test "$(nc-config --prefix)" = "$MPAS_PREFIX"
test "$(nf-config --prefix)" = "$MPAS_PREFIX"
test "$(pnetcdf-config --prefix)" = "$MPAS_PREFIX"

test -s "$NETCDF/include/netcdf.h"
test -s "$NETCDFF/include/netcdf.mod"
test -s "$PNETCDF/include/pnetcdf.h"

printf 'PIO=%s\n' "${PIO-<unset>}"
mpiexec --version
```

All three prefix comparisons must succeed. The header and module checks must also return without an error.

Restore the environment after logging in again

If you installed Open MPI and the libraries using this appendix, run this block in each new terminal or at the beginning of your batch job:

```
export MPAS_ROOT="$HOME/MPAS"
export MPAS_PREFIX="$MPAS_ROOT/opt/mpas-libs"
export MPI_PREFIX="$MPAS_ROOT/opt/openmpi-4.1.6"
export PATH="$MPI_PREFIX/bin:$MPAS_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPI_PREFIX/lib:$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"

export NETCDF="$(nc-config --prefix)"
export NETCDFF="$(nf-config --prefix)"
export PNETCDF="$(pnetcdf-config --prefix)"
unset PIO

command -v mpicc mpif90 mpirun mpiexec
nc-config --version
nf-config --version
pnetcdf-config --version
command -v ncdump gpmets
```

If you used the site's MPI module instead, load that same module rather than exporting the MPI_PREFIX paths above. Restore your library paths as well. You do not need to repeat any download or installation commands after logging in again.

After the dependency checks pass, return to Step 2 of Guide 1 to download and compile HKUST-MPAS.